Reproducing the accounting evidence

Study date: 19 October 2026. Integrated: 20 October 2026. Task: L001f, reproduction and consolidation. Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a fresh build reproduce the existing accounting results without using yesterday's binaries? The pre-run prediction is that all five evidence sets match their archived JSON exactly, every executable agrees with its independent expected outputs, and changing a saved cash value is detected. This follows L001e's boundary checks. Today consolidates one concept: reproducibility. There is no new trade feature, C++ syntax lesson, price series or empirical strategy experiment.

All inputs are the explicitly synthetic fixtures studied October 13-18. They have no market sample period or training/test split. The metric is exact equality of outputs, return status and saved evidence, with empty error output. We run the full reproduction twice, using fresh builds each time. We make no timing, speed, investment-return or personal-mastery claim.

Beginner explanation and vocabulary

Source code is the written recipe. A compiler turns it into an executable, or binary. A binary left in a build folder may come from an older recipe. Running it successfully would not show that today's source still works. A fresh build starts in an empty temporary directory and creates new executables from the source we intend to verify.

A fixture is a fixed input or expected result. An oracle computes the expected answer using a separate calculation. An archived result is a saved record of a previous run. These serve different purposes: the independent calculation checks arithmetic, while comparison with the archive checks whether the documented evidence still matches. Repeating a wrong calculation cannot make it right.

A hash is a compact fingerprint of bytes. Equal hashes help identify unchanged evidence; they do not prove a financial assumption or establish ownership of data. The recipe records SHA-256 of all five result files. The integration manifest separately records source and verifier inputs. A temporary directory keeps this exercise separate from existing build files and is removed afterward.

Reconstruct the ledger by hand

Let C mean cash in whole money units, q mean whole shares, and m mean a mark in money units per share. Holdings H = q*m and marked wealth W = C+H. A mark values a position; it is not a promise that the position can be sold at that price. Cash changes only when a cash event occurs.

Snapshot	Cash	Shares and mark	Marked wealth
Starting position	800	4 at 50	800+200 = 1000
Higher mark, no trade	800	4 at 55	800+220 = 1020
Lower mark, no trade	800	4 at 45	800+180 = 980

These are alternative snapshots of the same starting position, not three consecutive returns. No cash gain is realised merely because the mark rises. The C++ example multiplies shares by mark; the Python Decimal reference adds each marked share separately.

One buy, its fee and a rejection

Buy quantity n = 2 at fill price p = 50 with separate fee f = 3, starting from C = 800 and q = 4. Notional n*p = 100 is the amount exchanged for shares. Total cash debit D = n*p+f = 103. The accepted state is C_new = C-D = 697 and q_new = q+n = 6.

When the unchanged mark equals the fill, holdings rise from 200 to 300 and wealth becomes 697+300 = 997. Reconcile the two sides: cash falls 103, holdings rise 100, wealth falls 3. The 100 spent on shares is a transfer within wealth; the fee is the loss under this assumption. In symbols, C_new + q_new*p = C + q*p - f. This is an accounting identity, not an alpha result.

With starting cash 102, the same request must reject. Both cash 102 and shares 4 remain unchanged, and no fee is charged. With cash exactly 103 it accepts, leaving cash zero and six shares. This connects the hand example to yesterday's equality boundaries. Checking only the Boolean acceptance flag would miss a rejected request that secretly deducted a fee.

The reusable C++ try_buy contract caps cash, shares and fee at 10000 and requires positive quantity/price within that cap. It has no debt, short sales, fractional units or partial fills. Checks occur before mutation. Invalid starting state is rejected unchanged, not silently repaired. The separate struct snapshot and fixed equation programs are teaching slices, not a combined engine.

What the reproduction recipe does

From the repository root run: `.venv/bin/python scripts/reproduce_accounting.py`. The Python file is verification tooling; its syntax is not today's additional C++ lesson. It locates the repository, creates an empty temporary build directory and compiles marked_wealth.cpp and buy_equations.cpp with C++17, warnings enabled and warnings treated as errors.

It passes those fresh executable paths to their existing independent Decimal verifiers. It then runs the struct, validated-buy and boundary verifiers, each of which compiles its own fresh C++ program in temporary storage. Every verifier executes its C++ program twice and checks stdout, stderr and exit status. The two buy verifiers use UndefinedBehaviorSanitizer. The other three do not. No program is taken from the existing build directory; the child commands run from temporary storage.

Finally, each verifier's JSON must match its published archive byte for byte. An optional --evidence-dir selects scratch archives for the deliberate failure check; the originals stay intact. Two complete recipe executions produce identical result JSON: five fresh builds each, with two executions per build. This is output reproduction on the current toolchain, not identical binary bytes across platforms. The environment and compiler version remain part of the validation record.

Results and failure control

All five archived results reproduce exactly: three valuation snapshots, the one-buy equations, the initialized and empty structs, 14 fixed buy outcomes, and 8351 boundary comparisons. The latter contain 250 accepted and 8101 rejected unchanged outcomes. Do not combine these overlapping counts into a statistical sample size. The full make check passes 608 Python tests and all current C++ checks; dependency consistency and the recipe's lint/format checks also pass.

A scratch archive changes starting cash from 800 to 801 while retaining normal JSON formatting. The recipe exits with the intended archived-evidence mismatch. This tests the archive comparison, not a new bug in the unchanged C++ function. A real Python -O call is refused because assertions must remain enabled. Both correct full reproductions passed; no new dependency was installed.

Costs, assumptions and limits

The same-price wealth identity depends on fill equalling the unchanged mark. A different fill, spread, slippage or later mark changes the reconciliation. Today's recipe does not simulate those costs, execution delay, settlement, taxes, financing, market impact or uncertain fills. The specified whole-unit fee is paid only for an accepted buy; it is not a calibrated broker tariff.

The boundary grid is selected evidence, not every possible integer combination. The fixtures reset state between requests, so they do not establish correct sequences, timing or concurrency. A quiet sanitizer covers executed paths only. Yesterday's negative-price-guard mutant survives because a later affordability check still rejects; that limitation remains recorded rather than hidden.

The fixed C++ examples and Python references share the written contract and can share an incorrect assumption. Exact reproduction protects evidence continuity, not market realism. No price data was downloaded and no profitability claim is permitted. Q004a/Q005b licence and provenance gates remain.

Interview explanation and next step

I rebuild the accounting examples in temporary storage, verify their outputs against independent hand/Decimal or integer calculations, and compare the resulting evidence with saved records. I check return status and error output as well as values, and show that a changed archive fails. Passing supports reproducible bounded arithmetic. It does not establish an executable strategy or no leakage.

L001's repository teaching sequence is now locally reproduced and consolidated; this says nothing about Sidiq's personal review or mastery. Next learning is L002a: a hand timeline separating what is known when a decision is made from the later execution price. Do not jump to trend implementation. L002b-f, C003b, C004 returns/vectors and C005 statistics retain their separate prerequisite lessons.

The October 19 study was held locally after that day's one-commit allowance was used. Integrated October 20 after all 24 original inputs and seven artifacts matched their saved hashes and sizes. The recipe and result bytes are preserved; commands and publication prose now use published paths. Current reproduction, changed-archive and optimization controls are rerun before publication. Today's one-commit allowance funds this integration. The October 18 weekly synthesis remains queued. Personal review remains Not yet reviewed; no dates or reviews are backfilled. Continue at 08:00 Europe/London, including daylight saving, through December 31; then stop new research and request the next scope. No input blocks synthetic continuation.

Primary sources consulted on October 19

Python subprocess documents captured output, return codes, working directories and timeouts used by the reproduction harness: https://docs.python.org/3/library/subprocess.html

Clang documents UndefinedBehaviorSanitizer instrumentation and its checks: https://clang.llvm.org/docs/UndefinedBehaviorSanitizer.html

Microsoft documents integer multiplication and division; the cash/wealth identities above are project derivations: https://learn.microsoft.com/en-us/cpp/cpp/multiplicative-operators-and-the-modulus-operator?view=msvc-170

Check your understanding

1. Why is running yesterday's executable insufficient to validate today's source?

2. Why does a buy with notional 100 and fee 3 reduce wealth by 3 rather than 103 here?

3. What must remain unchanged when starting cash is 102 and the requested debit is 103?

4. Does exact reproduction establish that the decision used no future information?

Suggested answers

1. The binary may have been built from older source. Compile the intended source freshly and then check its output. Also record inputs and environment; a rebuild alone does not check correctness.

2. Cash falls 103 but marked shares rise 100. With fill equal to the unchanged mark, only the fee reduces total wealth. A different fill/mark relationship requires another calculation.

3. Both cash 102 and the original four shares, with no fee deduction. The rejection flag alone cannot establish unchanged state.

4. No. These independent snapshots and buys have no decision/execution timeline. L002 must define and test information availability, including a future-price mutation and a deliberately leaking control.